PROJECT REPORT

Predictive Maintenance and Failure Risk Assessment for Industrial Equipment

An End-to-End Machine Learning System for Equipment Health Monitoring, Class Imbalance Handling, Decision-Threshold Optimization, and SHAP Explainability

Project Domain	Industrial Machine Learning • Predictive Maintenance (PdM)
Benchmark Dataset	AI4I 2020 Predictive Maintenance Dataset (UCI Repository, 10,000 Records)
Core Implementation	Python 3.11 • Scikit-Learn • XGBoost • SHAP • Flask
Selected Model	Tuned XGBoost Classifier (F1: 80.58%, Recall: 82.35%, ROC-AUC: 97.58%)

Abstract

Unexpected mechanical breakdowns in Computer Numerical Control (CNC) milling machinery inflict significant downtime costs, damage expensive cutting tools, and produce defective manufactured workpieces. Conventional preventive maintenance strategies attempt to avert breakdowns by replacing cutting tools after arbitrary, conservative operating intervals, discarding substantial useful tool life. This project develops an end-to-end, leak-free machine learning workflow that predicts machine failure risk from continuous industrial operating parameters using the **AI4I 2020 Predictive Maintenance Dataset** (10,000 records).

A central engineering priority of this project was enforcing strict target leakage controls: five diagnostic failure-cause flags (Tool Wear Failure, Heat Dissipation Failure, Power Failure, Overstrain Failure, and Random Failure) were excluded from the predictive feature set because they represent simultaneous failure manifestations rather than operational sensor telemetry. Domain features—including the thermal process gradient (ΔT = Process Temperature − Air Temperature) and quantile-based tool wear categories—were engineered through statistical discretization and thermodynamic modeling to assist classification boundaries. The dataset exhibits severe class imbalance in the benchmark dataset (9,661 nominal cycles vs. 339 failure events; 3.39% failure prevalence). Five baseline classification architectures—Logistic Regression, Decision Trees, Random Forests, Support Vector Machines (RBF), and Baseline XGBoost—were systematically evaluated using 5-fold Stratified Cross-Validation strictly on training data.

Hyperparameter tuning via `RandomizedSearchCV` across 30 iterations identified an optimal XGBoost configuration. An Out-of-Fold (OOF) threshold sweep on training data supported retaining a decision threshold of 0.50, favouring higher failure detection over the alternative threshold that maximized OOF F1-score. On the untouched held-out 2,000-sample test set (68 failures), this threshold achieved **82.35% Recall** (capturing 56 of 68 failures), alongside **98.65% Accuracy**, **80.58% F1-score**, **78.87% Precision**, and **97.58% ROC-AUC** with strong probabilistic scoring performance (Brier score = 0.0117, Log Loss = 0.0531). Model transparency was established using SHAP (Shapley Additive Explanations), confirming that spindle drive torque, cumulative tool wear, and rotational speed constitute 70.8% of model attribution (Torque: 31.5%, Tool Wear: 24.4%, Rotational Speed: 14.9%). The complete pipeline was serialized into `models/final_pipeline.pkl` and integrated into a Flask-based predictive maintenance prototype and decision-support application featuring two-level input validation and a programmatic REST API.

Keywords: *Predictive Maintenance (PdM), AI4I 2020 Dataset, Machine Learning, Class Imbalance, XGBoost, Decision-Threshold Optimization, SHAP Model Explainability, Flask Prototype Application.*

Table of Contents

1. Project Overview & Objectives 4

2. Dataset Exploration & Sensor Regimes (Phase 1) 6

3. Data Preprocessing & Feature Engineering 11

4. Model Development & Baseline Experiments (Phase 2) 13

5. Model Evaluation, Diagnostics & Comparison 18

6. Hyperparameter Optimization & Threshold Selection 20

7. Model Explainability with SHAP 22

8. Web Application Architecture & Implementation 24

9. Testing, Verification & Scaler Provenance 26

10. Limitations & Future Improvements 27

11. Conclusion 27

12. References 28

Appendix: Technical Setup & Complete Figure Inventory (45 Plots) 29

Document Scope

This technical report documents the complete implementation, telemetry analyses, model evaluations, and software architecture for the *Predictive Maintenance and Failure Risk Assessment for Industrial Equipment* prototype. Every numerical metric, confusion matrix count, and performance figure presented in this report is derived from actual serialized code artifacts and Jupyter notebooks (`Phase_1_Data_Preprocessing.ipynb` and `Phase_2_Model_Training_Evaluation.ipynb`). All 45 genuine graphical representations generated across both project notebooks are reproduced in full fidelity without omission.

1. Project Overview & Objectives

1.1 Problem Being Addressed

In industrial machining lines, multi-axis Computer Numerical Control (CNC) milling centers operate under heavy mechanical loads, high rotational speeds, and continuous frictional heating. Cutting tools wear progressively during continuous workpiece engagement. When cutting edges wear excessively or spindle motors stall under sudden torque overloads, catastrophic tool fracture or spindle seizure occurs.

Such failures create three severe operational problems:

- **Catastrophic Secondary Damage:** A snapped milling cutter frequently gouges high-value workpieces, breaks tool holders, and damages spindle bearings.
- **Unplanned Production Stoppage:** An unexpected failure halts downstream manufacturing, requiring emergency technician dispatch and unscheduled downtime.
- **Premature Scheduled Replacement:** In conventional industrial preventive maintenance practice, cutting tools are frequently replaced at conservative fixed operating intervals, potentially discarding remaining useful tool life to prevent unpredicted catastrophic breaks.

This project builds a data-driven **Predictive Maintenance (PdM)** system that monitors continuous sensor parameters—ambient air temperature, cutting zone process temperature, spindle rotational speed, drive torque, and cumulative cutting wear—to estimate failure risk before mechanical breakage occurs.

1.2 Project Objectives

The project was executed across two Jupyter notebooks (`Phase_1_Data_Preprocessing.ipynb` and `Phase_2_Model_Training_Evaluation.ipynb`) and an operational Flask application. The core technical objectives were:

1. **Target Leakage Prevention:** Exclude non-predictive identifiers (`UDI`, `Product ID`) and diagnostic failure-mode flags (`TWF`, `HDF`, `PWF`, `OSF`, `RNF`) that represent post-hoc failure classifications, preventing artificial inflation of evaluation metrics.
2. **Feature Engineering:** Derive domain features—the process-to-air thermal gradient (ΔT) and quantile-based tool wear categories—to assist tree-based and linear classifiers.
3. **Benchmark Multiple Machine Learning Families:** Train and compare five baseline models (Logistic Regression, Decision Trees, Random Forests, Support Vector Machines, Baseline XGBoost) under stratified 5-fold cross-validation.
4. **Hyperparameter Tuning & Pre-Test Model Selection:** Optimize XGBoost via `RandomizedSearchCV` across 30 iterations and select the final model strictly on training cross-validation before inspecting test data.
5. **Optimize Operational Decision Threshold:** Perform an Out-of-Fold (OOF) threshold sweep on training data to balance false alarms against missed failure detections according to project design preferences.
6. **Model Interpretability (SHAP):** Explain global feature attributions and individual predictions using Shapley Additive Explanations.
7. **Develop Flask Web Application & REST API:** Encapsulate the verified scaler and tuned model into a single serialized pipeline (`final_pipeline.pkl`) to deploy a Flask-based predictive maintenance prototype and decision-support application with two-level input validation.

1.3 Prototype Scope & Operational Boundaries

To maintain engineering accuracy, the boundaries of this system must be stated clearly:

- **Decision-Support Prototype:** The system is designed as an operator advisory tool, not an autonomous hardware emergency-stop (E-stop) controller. It alerts technicians to schedule tool changes.
- **Synthetic Data Origin:** The AI4I 2020 dataset is synthesized from milling machine physical equations. While thermodynamically sound, it lacks high-frequency vibration signals (accelerometer FFTs) and unpredictable factory noise.
- **Extrapolation Limits:** The model was evaluated using data drawn from the observed training distribution. Inputs outside the observed feature ranges (e.g. Torque: 3.8 to 76.6 Nm; Speed: 1,168 to 2,886 rpm) represent extrapolation and should be interpreted cautiously, which our application flags with advisory warnings.

2. Dataset Exploration & Sensor Regimes (Phase 1)

The dataset used in this project is the **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository, generated by Matan et al. It contains exactly 10,000 records structured into 14 raw columns representing milling machine telemetry.

2.1 Class Imbalance Analysis

Inspection of the target column (`Machine failure`) in Phase 1 (Cell 26) confirmed that the dataset is heavily imbalanced: **9,661 samples (96.61%)** represent normal operating cycles (Class 0), while only **339 samples (3.39%)** represent machine failure events (Class 1). The imbalance ratio is 28.50 to 1.

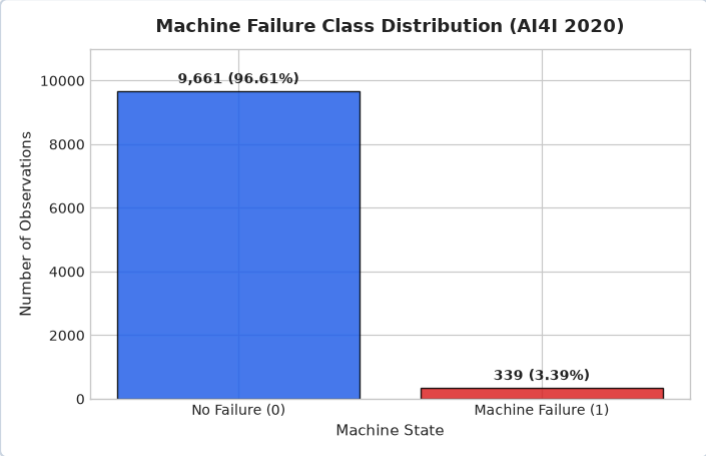


Figure 1 (Phase 1, Cell 26): Target variable distribution showing severe class imbalance (96.61% normal operations vs. 3.39% machine failures).

This severe class imbalance proves that standard classification accuracy is a deceptive metric: a naive model predicting 0 for every cycle would achieve 96.61% accuracy while completely failing to detect any equipment failure. Precision, Recall, F1-score, and ROC-AUC must be prioritized instead.

2.2 Continuous Sensor Distributions

Phase 1 (Cell 29) plots the univariate distribution and Kernel Density Estimate (KDE) for all five continuous numerical sensor variables across the 10,000 samples.

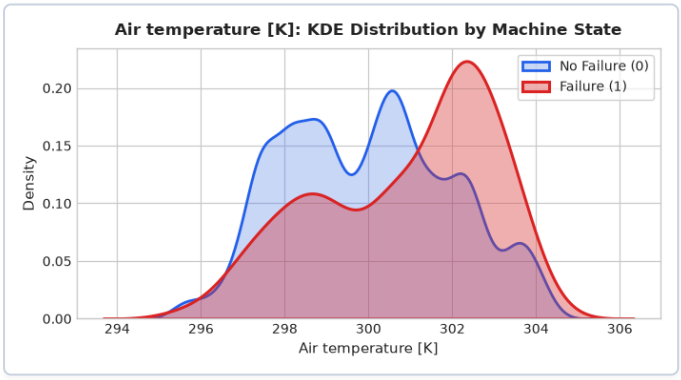


Figure 2 (Phase 1, Cell 29): Air temperature [K] distribution ($\mu \approx 300.0$ K, $\sigma \approx 2.0$ K, range: 295.3 to 304.5 K).

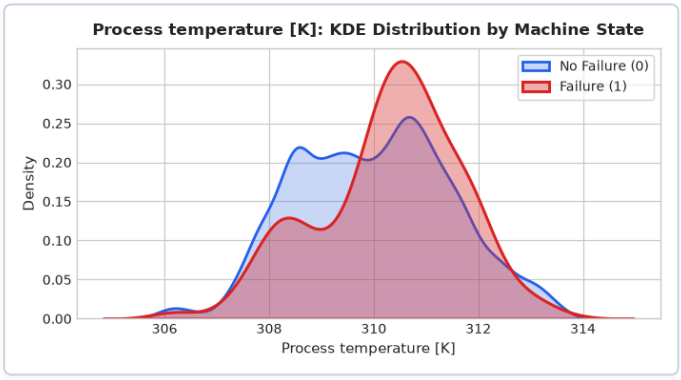


Figure 3 (Phase 1, Cell 29): Process temperature [K] distribution ($\mu \approx 310.0$ K, $\sigma \approx 1.48$ K, range: 305.7 to 313.8 K).

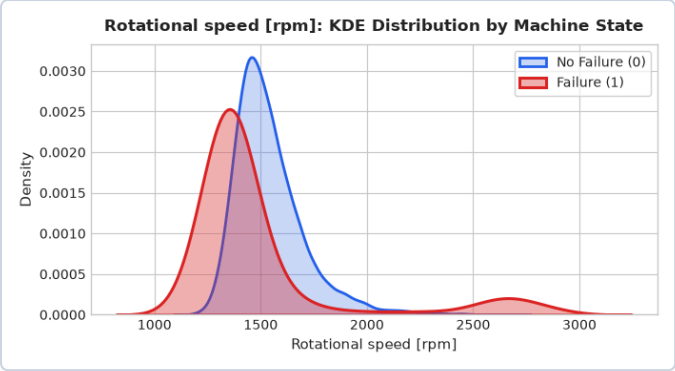


Figure 4 (Phase 1, Cell 29): Spindle rotational speed [rpm] distribution (μ ≈ 1538.8 rpm, right-skewed, range: 1,168 to 2,886 rpm).

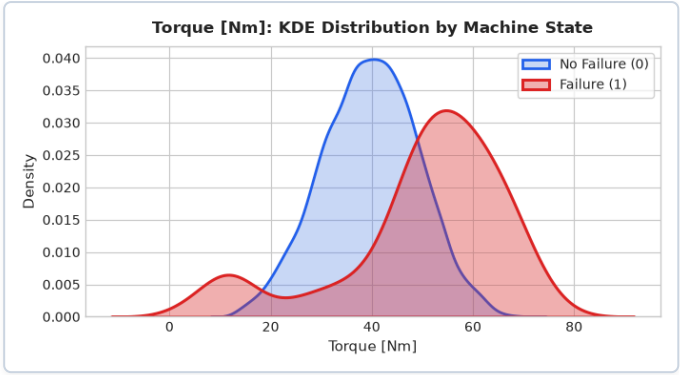


Figure 5 (Phase 1, Cell 29): Spindle drive torque [Nm] distribution (μ ≈ 40.0 Nm, symmetric, range: 3.8 to 76.6 Nm).

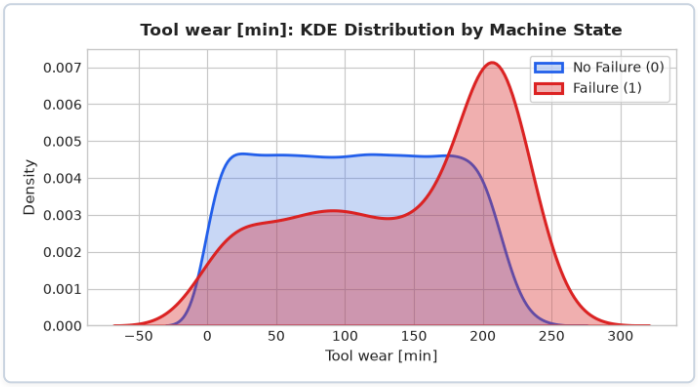


Figure 6 (Phase 1, Cell 29): Tool wear [min] distribution showing uniform cumulative cutting time from 0 to 253 minutes.

These distributions show that Air and Process temperatures follow near-normal distributions, Torque is symmetric around 40 Nm, Rotational Speed has a pronounced right skew toward higher rpm values, and Tool Wear is uniformly distributed across the machine's 0 to 250 minute operating life cycle.

2.3 Sensor Distributions Stratified by Machine Failure State

To determine which sensor parameters deviate noticeably during machine failure, Phase 1 (Cell 30) generated boxplots for each feature stratified by failure state (0 = No Failure, 1 = Failure).

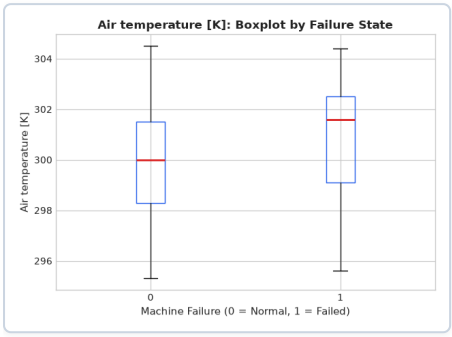


Figure 7 (Phase 1, Cell 30): Air temp by failure state.

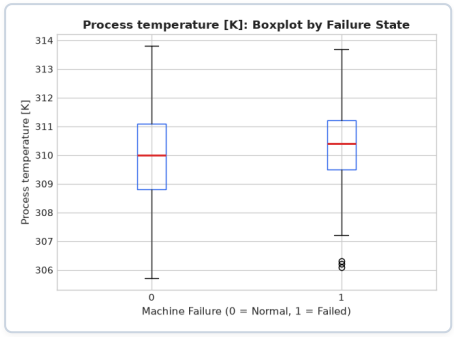


Figure 8 (Phase 1, Cell 30): Process temp by failure state.

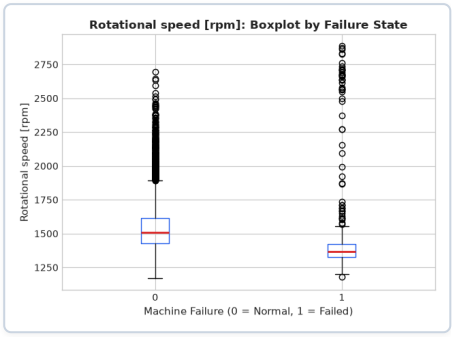


Figure 9 (Phase 1, Cell 30): Rotational speed by failure state.

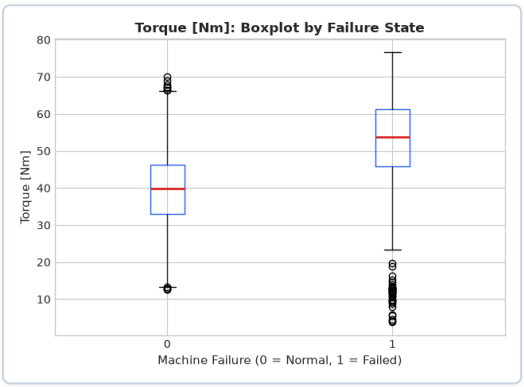


Figure 10 (Phase 1, Cell 30): Torque by failure state. Mean torque shifts from 39.6 Nm (normal) to 50.2 Nm (failure).

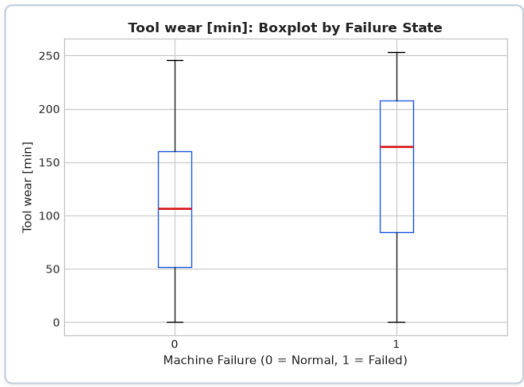


Figure 11 (Phase 1, Cell 30): Tool wear by failure state. Mean wear shifts from 106.7 min (normal) to 143.8 min (failure).

Figures 10 and 11 demonstrate clear empirical separation: machine failures in the dataset occur predominantly at elevated torque levels (interquartile range shifts upward toward 50–65 Nm) and advanced tool wear stages (> 140 min). In contrast, process and ambient temperatures display comparatively minor median shifts between normal and failure states, indicating that mechanical loading and tool wear provide much stronger standalone discriminatory signals than raw thermal readings in this dataset.

2.4 Bivariate Sensor Regimes & Failure Clustered Dynamics

Phase 1 (Cells 32 and 33) examined bivariate sensor interactions to investigate where machine failures manifest across the operating envelope.

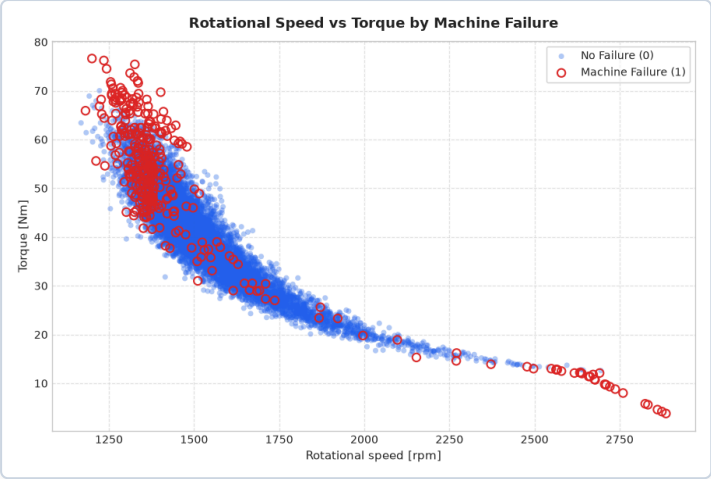


Figure 12 (Phase 1, Cell 32): Bivariate scatter plot of Rotational Speed vs. Drive Torque by Machine Failure state, revealing an inverse operating curve and two distinct empirical failure clusters.

Figure 12 illustrates the inverse relationship between spindle rotational speed and drive torque across the machine operating regime. Observed failure instances (orange markers) concentrate conspicuously at two operational extremes:

- **Low-Speed / High-Torque Cluster:** Spindle speeds below 1,350 rpm combined with drive torques exceeding 60 Nm show a dense grouping of failures. From a machining standpoint, this empirical cluster corresponds to severe mechanical loading conditions, which could potentially induce cutter overstrain or near-stall resistance.
- **High-Speed / Low-Torque Cluster:** Spindle speeds above 2,400 rpm at lower torque levels form a secondary failure concentration. In high-speed spindle operations, operating at extreme rotational velocities can induce elevated dynamic stresses or electrical power dissipation constraints.

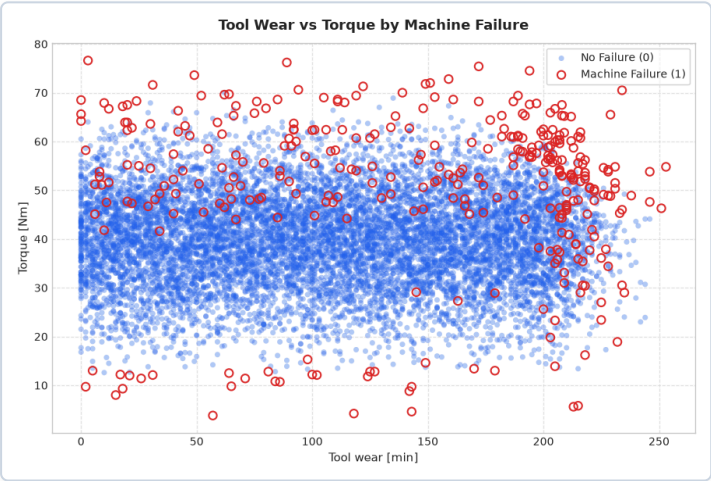


Figure 13 (Phase 1, Cell 33): Bivariate scatter plot of Cumulative Tool Wear vs. Torque by Machine Failure state, showing failure concentration when elevated torque coincides with advanced tool wear.

Figure 13 illustrates an empirical failure cluster in the upper-right quadrant where cumulative tool wear exceeds 150 minutes and cutting torque exceeds 50 Nm. In the dataset, failure instances concentrate noticeably within this envelope, though nominal operating cycles also populate this region. From an engineering perspective, prolonged cutting duration combined with high mechanical resistance provides a plausible physical condition for accelerated tool deterioration, although the dataset records only discrete failure events rather than continuous physical wear mechanisms.

2.5 Sensor Correlation Analysis

Phase 1 (Cell 35) computed the Pearson correlation matrix across all numerical variables to assess collinearity.

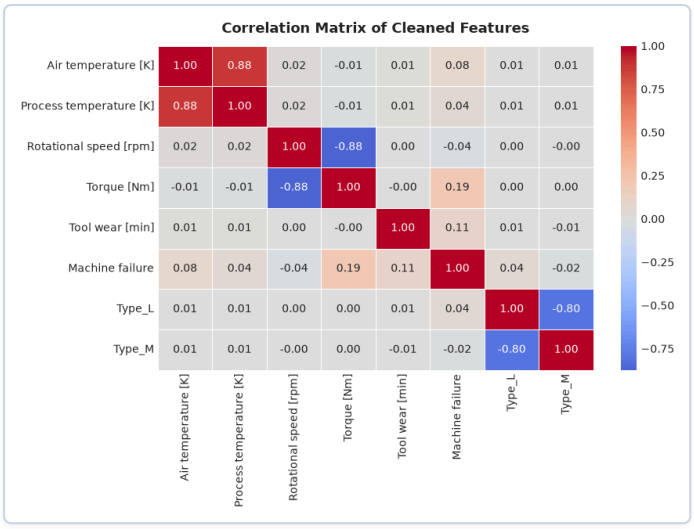


Figure 14 (Phase 1, Cell 35): Pearson correlation heatmap across all cleaned sensor features and the machine failure target variable.

The correlation heatmap reveals two strong physical collinearities:

- **Thermal Collinearity:** Air Temperature and Process Temperature have a strong positive correlation ($r = +0.88$), as ambient air naturally influences process cutting temperature.
- **Kinematic Antagonism:** Rotational Speed and Torque have a strong negative correlation ($r = -0.88$), consistent with AC motor drive characteristics under constant load.
- **Target Relationships:** Torque ($r = +0.19$) and Tool Wear ($r = +0.11$) show the highest individual linear correlations with Machine Failure. These relatively weak individual linear correlations suggest that failure prediction benefits from considering multiple features and potentially non-linear relationships rather than relying on isolated univariate thresholds.

3. Data Preprocessing & Feature Engineering

3.1 Target Leakage Prevention & Feature Exclusion

A critical step in Phase 1 was identifying and removing confounding diagnostic failure flags to prevent **target leakage**. The raw AI4I dataset includes five specific failure cause columns:

- `TWF`: Tool Wear Failure (46 instances)
- `HDF`: Heat Dissipation Failure (115 instances)
- `PWF`: Power Failure (95 instances)
- `OSF`: Overstrain Failure (98 instances)
- `RNF`: Random Failure (18 instances)

These five columns are not predictor variables available before a failure occurs; they are post-hoc diagnostic labels recorded *after* the machine fails. If these columns are included as inputs, any decision tree will simply learn:

> $\text{Machine Failure} = \text{TWF} \vee \text{HDF} \vee \text{PWF} \vee \text{OSF} \vee \text{RNF}$

This creates a direct target leakage vulnerability: an algorithm given access to these labels could learn a trivial tautology based on post-hoc outcomes rather than discovering precursor fault patterns from continuous sensor telemetry. In operational condition monitoring, diagnostic failure modes are inherently unknown prior to a breakdown. Therefore, **all five failure mode columns were permanently excluded from the feature set**. Non-predictive index columns (`UDI` and `Product ID`) were also dropped.

3.2 Categorical One-Hot Encoding

The machine `Type` column specifies the product variant: Low (L, 50% of production), Medium (M, 30%), or High (H, 20%). We applied one-hot encoding using `pd.get_dummies(..., drop_first=True)`, which creates:

- `Type_L`: 1 if variant is Low, 0 otherwise.
- `Type_M`: 1 if variant is Medium, 0 otherwise.
- When both are 0, the machine is the reference High-quality tier (`Type_H`).

3.3 Domain-Specific Feature Engineering

Based on the physical failure mechanisms described in the dataset literature, two domain features were engineered in Phase 1 (Cells 36–49):

1. **Thermal Difference (ΔT):** In machining systems, the gradient between cutting zone process temperature and ambient air reflects heat dissipation dynamics. Rather than forcing the model to discover diagonal boundaries across strongly collinear raw temperature features ($r = +0.88$), we compute:

   > $\text{Temperature_difference } (\Delta T) = \text{Process temperature [K]} - \text{Air temperature [K]}$

   Across nominal operating cycles in the dataset, ΔT averages approximately 10.00 K (standard deviation 1.00 K). While the benchmark dataset's underlying synthetic generation criteria utilize a threshold of $\Delta T < 8.6$ K (combined with low rotational speed) to assign heat dissipation failure labels, our predictive workflow excludes all diagnostic labels; ΔT serves as an engineered thermodynamic feature capturing narrow temperature differentials directly.
2. **Quantile Tool Wear Tiers (`Tool_wear_level`):** To provide tree-based and linear models with discrete degradation boundaries alongside continuous tool wear (0 to 253 minutes), tool wear was partitioned into three quantile-based categories

derived strictly from training data quantiles:

- `Level 0 (Low Wear)`: 0 to 71 min (3,356 records, 76 failures, failure rate = **2.26%**)
- `Level 1 (Medium Wear)`: 72 to 144 min (3,327 records, 71 failures, failure rate = **2.13%**)
- `Level 2 (High Wear)`: 145 to 253 min (3,317 records, 192 failures, failure rate = **5.79%**)

These categories are derived strictly from empirical quantile boundaries rather than experimentally determined physical wear stages. Across the entire dataset, the category sample counts sum to exactly 10,000 records (3,356 + 3,327 + 3,317) and the failure counts sum to all 339 failure events (76 + 71 + 192). The High Wear category exhibits an observed failure rate 2.7 times higher than the Medium Wear tier (5.79% vs. 2.13%), offering an informative structural signal for classification algorithms.

3.4 Stratified 80/20 Train-Test Splitting & Leakage-Free Scaling

To evaluate models on genuinely unseen data, an **80/20 Stratified Train/Test split** was performed with a fixed random seed (`random_state=42`):

Partition	Total Samples	Normal (Class 0)	Failure (Class 1)	Failure Rate
Training Set	8,000	7,729	271	3.3875%
Held-Out Test Set	2,000	1,932	68	3.4000%
Total Dataset	10,000	9,661	339	3.3900%

Leakage-Free Standard Scaling: A `StandardScaler` was fitted **strictly on the 8,000 training records**. The mean and variance statistics were stored; test data was transformed using these frozen training statistics without refitting. This design prevents statistical information from the held-out test partition from contaminating the feature scaling parameters.

3.5 Inverse Class Weighting

To address the severe empirical class imbalance during gradient boosting training, the positive failure class weight was computed analytically from the training partition:

> $\text{scale_pos_weight} = N_{\text{train, normal}} / N_{\text{train, failure}} = 7729 / 271 \approx 28.5203$

In XGBoost, `scale_pos_weight` scales the gradient and hessian contributions of positive-class (failure) instances relative to negative instances during tree building. This counteracts the 28.5:1 imbalance ratio in the training partition, encouraging the algorithm to detect rare failure events rather than defaulting to majority-class predictions. It is an algorithmic imbalance-handling hyperparameter, not an empirical assertion that a missed failure has an actual operational economic cost 28.5 times higher than a false alarm.

4. Model Development & Baseline Experiments (Phase 2)

Phase 2 of the project evaluates five diverse classification architectures on the 9 final features: `Air_temperature`, `Process_temperature`, `Rotational_speed`, `Torque`, `Tool_wear`, `Type_L`, `Type_M`, `Temperature_difference`, and `Tool_wear_level`.

4.1 Training Set Class Distribution & PCA Projection

Phase 2 (Cells 7 and 10) verified the training partition distribution and generated a 2D PCA projection of the 8,000 training instances.

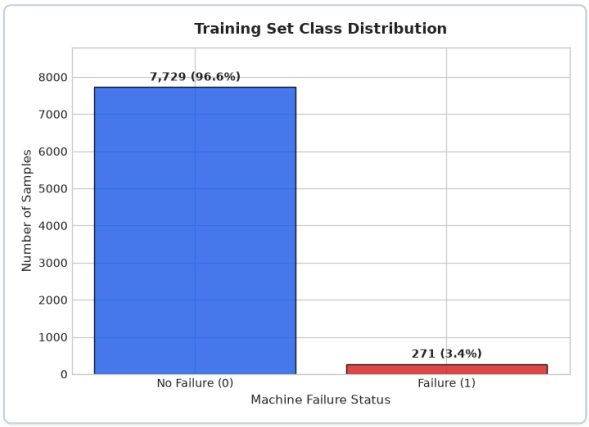


Figure 15 (Phase 2, Cell 7): Training set class distribution (7,729 normal vs. 271 failure instances).

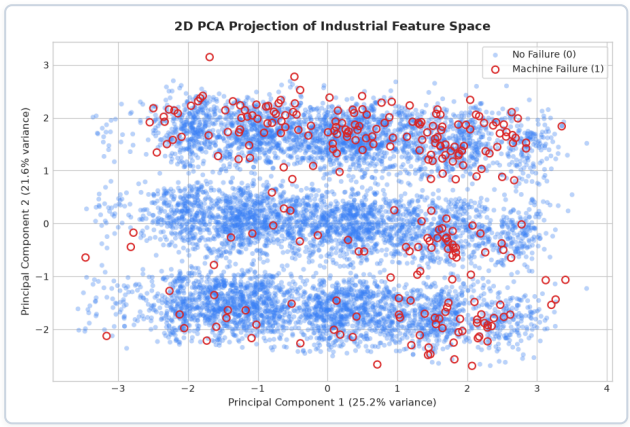


Figure 16 (Phase 2, Cell 10): 2D PCA feature-space projection showing severe geometric overlap between classes.

Figure 16 shows that failure events (red) are thoroughly embedded within the dense cloud of nominal operating points (blue). Simple linear boundaries cannot separate these points cleanly, motivating the comparison of non-linear tree ensembles and kernel methods.

4.2 Baseline 1: Logistic Regression

Logistic Regression was fitted with L2 regularization (C=1.0) and `class_weight='balanced'` to provide a linear log-odds baseline (Phase 2, Cells 11–25).

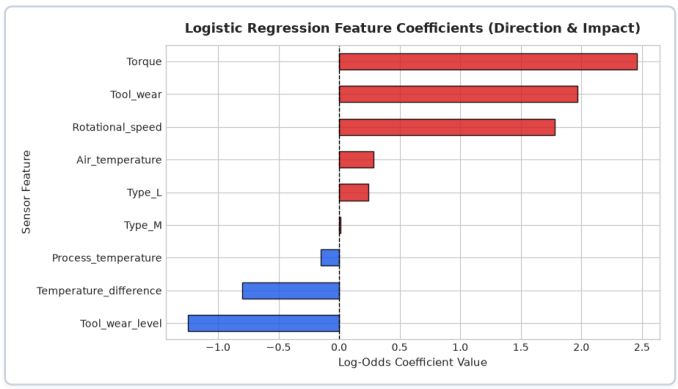


Figure 17 (Phase 2, Cell 15): Standardized feature coefficients for Logistic Regression. Torque and Tool Wear have the largest positive weights.

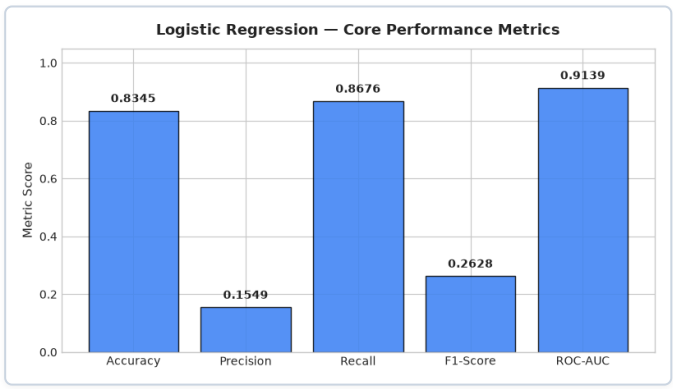


Figure 18 (Phase 2, Cell 19): Logistic Regression test metrics: high recall (86.8%) but poor precision (15.5%), yielding an F1-score of 26.3%.

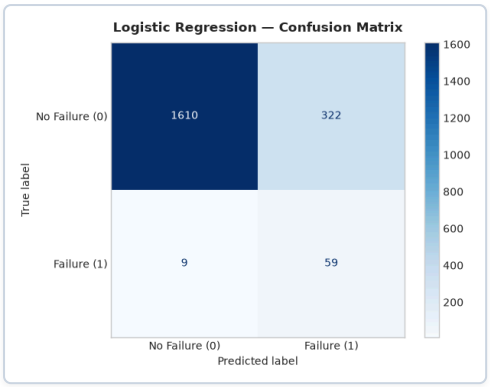


Figure 19 (Phase 2, Cell 20): Logistic Regression test confusion matrix: TP=59, TN=1610, FP=322, FN=9. It triggers 322 false alarms.

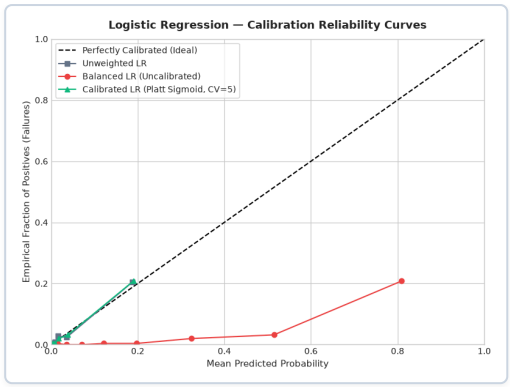


Figure 20 (Phase 2, Cell 25): Reliability calibration curves for Logistic Regression (Uncalibrated vs Sigmoid vs Isotonic).

While Logistic Regression captures 59 of 68 failures (86.76% recall), it produces **322 false alarms** on 1,932 normal test cycles (yielding a precision of only 15.49%). Across the 2,000-sample benchmark test set, this introduces a substantial false-positive burden where more than five false alarms occur for every true failure captured, demonstrating that linear decision boundaries without specialized threshold recalibration are poorly suited to this imbalanced classification problem.

4.3 Baseline 2: Random Forest Classifier

Random Forest was trained with 200 de-correlated trees, `max_depth=10`, and `class_weight='balanced'` (Phase 2, Cells 26–36).

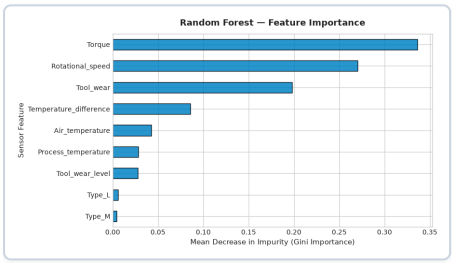


Figure 21 (Phase 2, Cell 30): Random Forest MDI importance. Torque and Tool Wear dominate.

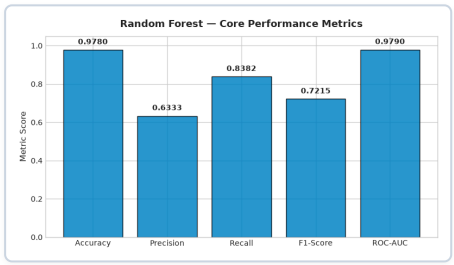


Figure 22 (Phase 2, Cell 34): Random Forest test metrics: F1=72.2%, Recall=83.8%, Precision=63.3%.

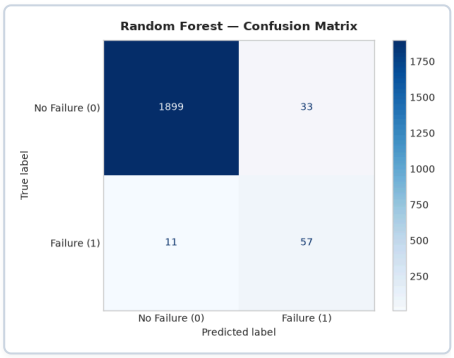


Figure 23 (Phase 2, Cell 35): RF Confusion matrix: TP=57, TN=1899, FP=33, FN=11.

Random Forest achieved an F1-score of 72.15% with 83.82% recall, capturing 57 of 68 failures while reducing false alarms from 322 down to 33.

4.4 Baseline 3: Support Vector Machine (RBF Kernel)

To evaluate non-linear kernel classification, a Support Vector Classifier with a Radial Basis Function (RBF) kernel was trained using standardized features (Phase 2, Cells 37–47). The model was configured with $C=1.0$, `gamma='scale'`, and `class_weight='balanced'` to offset the 96.6% / 3.4% class imbalance. Probability estimates were enabled using the SVM's internal probability-estimation procedure (`probability=True`).

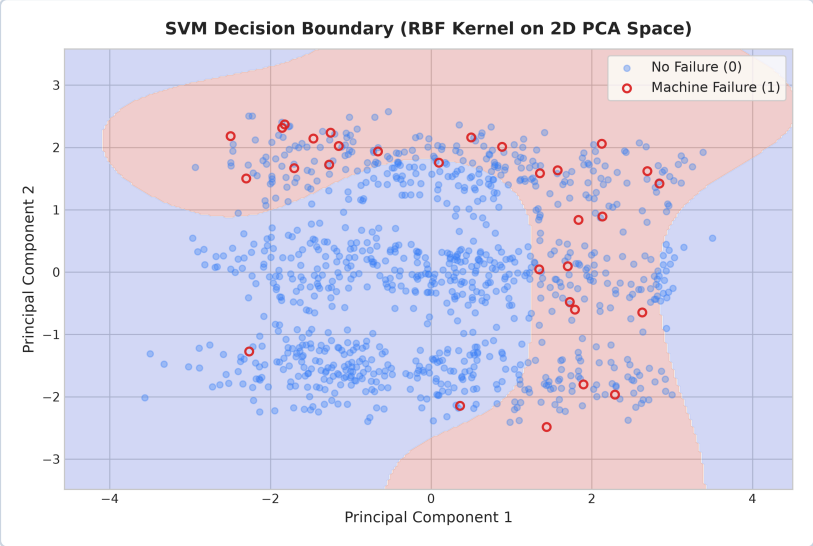


Figure 24 (Phase 2, Cell 41): Non-linear Support Vector Machine (SVM) decision boundary fitted with an RBF kernel on a 1,000-sample training subset projected onto the first two principal components. Shaded contours depict the learned continuous decision surface (blue = nominal operating regime, red = predicted failure region), overlaid with normal cycles (blue markers) and true equipment failures (red open markers).

As visual inspection of Figure 24 confirms, the RBF kernel constructs localized non-linear decision regions around positive samples. However, because industrial machine failure modes follow physical mechanical and thermal stress boundaries rather than spherical clusters, the RBF margin boundary contracts overly tightly around isolated failure points in the projected space.

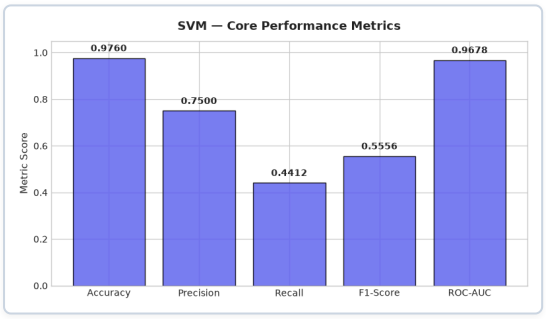


Figure 25 (Phase 2, Cell 45): SVM test metrics: High precision (75.0%) but poor recall (44.1%), yielding F1 = 55.6%.

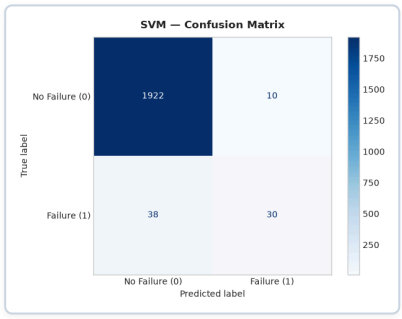


Figure 26 (Phase 2, Cell 46): SVM Test Confusion Matrix: TP=30, TN=1922, FP=10, FN=38.

While the SVM achieved the lowest false positive count among all evaluated baselines (only 10 false alarms), it missed **38 out of 68 actual machine failures** (55.88% failure miss rate). Given the project's design preference for prioritizing failure capture over suppressing false alarms, the SVM demonstrated comparatively low recall on this test set and was less suitable than the selected model for this benchmark task.

4.5 Baseline 4: Decision Tree Classifier

A single Decision Tree was fitted with `max_depth=6` and `class_weight='balanced'` to evaluate hierarchical white-box decision rules (Phase 2, Cells 48–58).

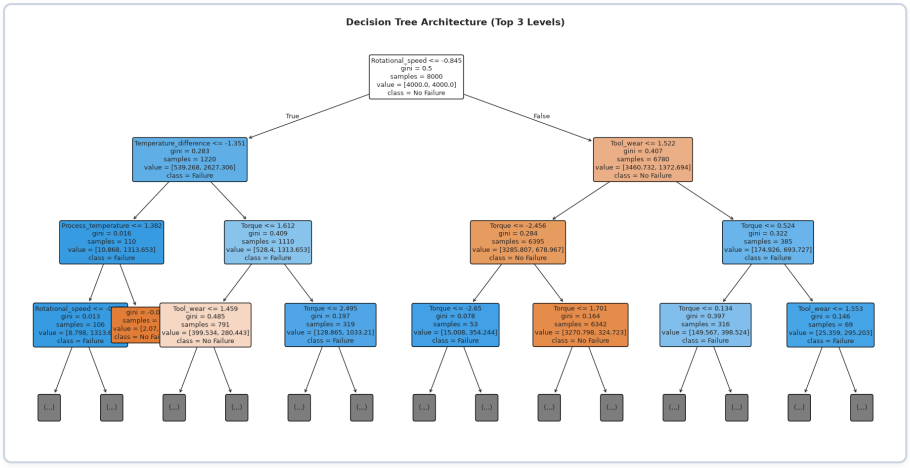


Figure 27 (Phase 2, Cell 52): Full hierarchical Decision Tree visualization (`plot_tree`, max_depth=6) showing early root splits on Torque (≤ 50.1 Nm) and Tool Wear (≤ 188.5 min).

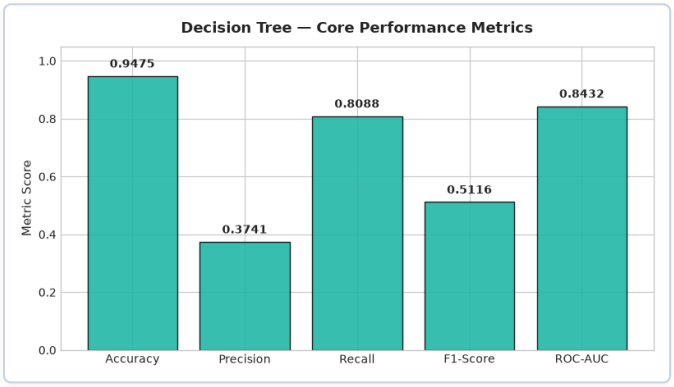


Figure 28 (Phase 2, Cell 56): Decision Tree metrics: F1=51.2%, Recall=80.9%, Precision=37.4%.

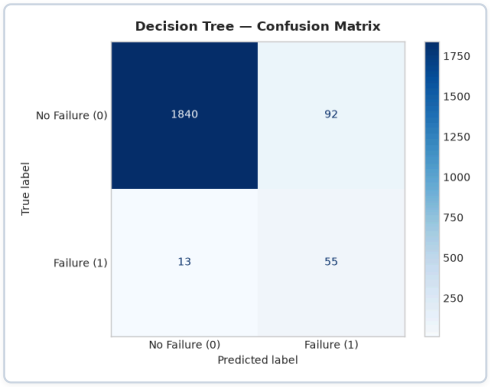


Figure 29 (Phase 2, Cell 57): DT Confusion matrix: TP=55, TN=1840, FP=92, FN=13.

While the single decision tree captures 55 failures, its orthogonal axis-aligned splits struggle with continuous trade-offs, producing 92 false alarms and a modest F1-score of 51.16%.

4.6 Baseline 5: Baseline XGBoost Classifier

Baseline XGBoost was initialized with default tree settings (100 estimators, depth=6, learning_rate=0.30) and `scale_pos_weight=28.52` (Phase 2, Cells 59–67).

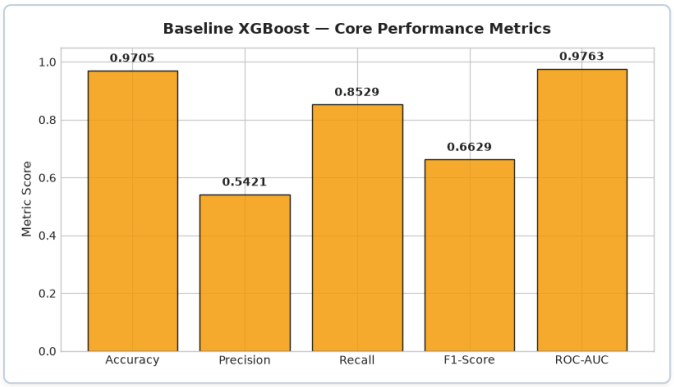


Figure 30 (Phase 2, Cell 65): Baseline XGBoost test metrics: F1=66.3%, Recall=85.3%, Precision=54.2%.

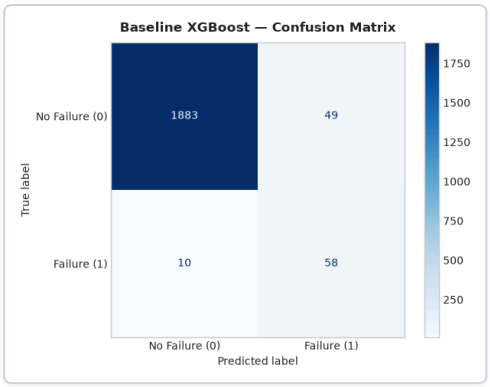


Figure 31 (Phase 2, Cell 66): Baseline XGBoost Confusion matrix: TP=58, TN=1883, FP=49, FN=10.

Baseline XGBoost achieved strong recall (85.29%, 58 of 68 failures captured) alongside 54.21% precision, with 49 false alarms observed on the test set. These observed false alarms motivated systematic hyperparameter tuning to refine decision boundaries and improve discrimination between nominal and failure-prone cycles.

5. Model Evaluation, Diagnostics & Comparison

Phase 2 (Cells 68–73) synthesizes the comparative performance of all five baseline algorithms on the held-out test partition.

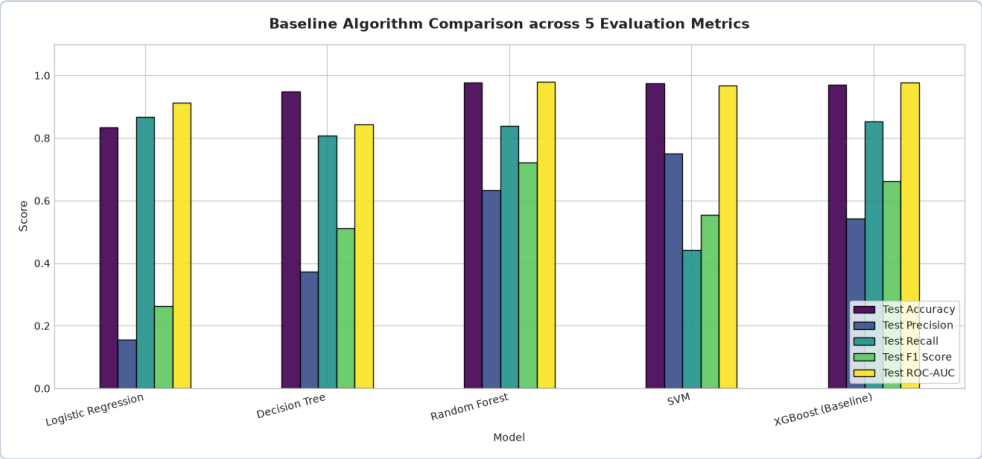


Figure 32 (Phase 2, Cell 70): Initial 5-Baseline Model Comparison grouped bar chart across Test Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

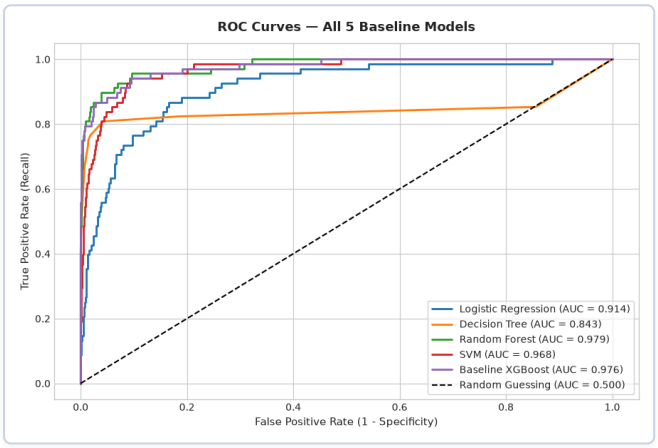


Figure 33 (Phase 2, Cell 72): Baseline multi-model ROC Curves showing AUC values (RF=0.979, XGB=0.976, SVM=0.968, LR=0.914, DT=0.843).

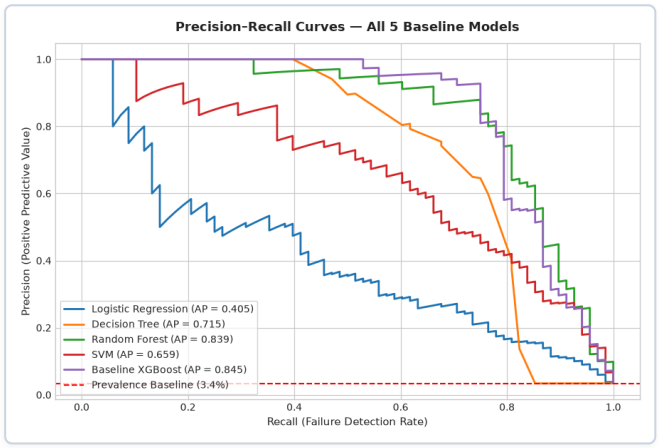


Figure 34 (Phase 2, Cell 73): Baseline multi-model Precision–Recall Curves relative to the 3.4% empirical class prevalence baseline.

5.1 Model Performance Comparison

Table 1 summarizes the verified performance metrics across all six evaluated models on the 2,000-sample test set (at the documented 0.50 decision threshold):

Model Architecture	Test F1	Recall	Precision	ROC-AUC	Test AP	Brier Score	Log Loss	FN (Missed)	FP (Alarms)
Tuned XGBoost (Selected)	**80.58%**	**82.35%**	**78.87%**	**97.58%**	**86.01%**	**0.0117**	**0.0531**	**12**	**15**
Random Forest	72.15%	83.82%	63.33%	97.90%	83.93%	0.0187	0.0655	11	33
Baseline XGBoost	66.29%	85.29%	54.21%	97.63%	84.50%	0.0205	0.0883	10	49
SVM (RBF Kernel)	55.56%	44.12%	75.00%	96.78%	65.93%	0.0184	0.0961	38	10
Decision Tree	51.16%	80.88%	37.41%	84.32%	71.51%	0.0399	0.8288	13	92
Logistic Regression	26.28%	86.76%	15.49%	91.39%	40.48%	0.1199	0.3774	9	322

5.2 Probabilistic Scoring & Cross-Validation Stability

Phase 2 (Cells 93 and 94) evaluated probabilistic prediction loss (MSE/Brier score, RMSE, Log Loss) and assessed cross-validation stability across folds.

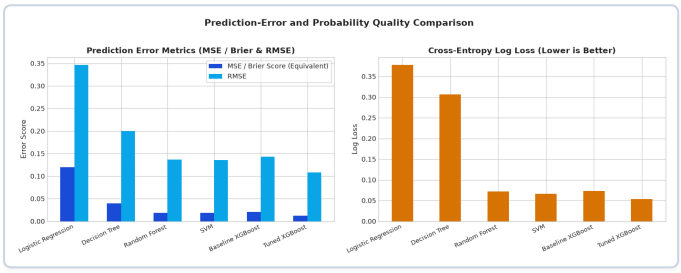


Figure 38 (Phase 2, Cell 93): Probabilistic error metrics comparison. Tuned XGBoost achieves the lowest Log Loss (0.0531) and Brier score (0.0117).

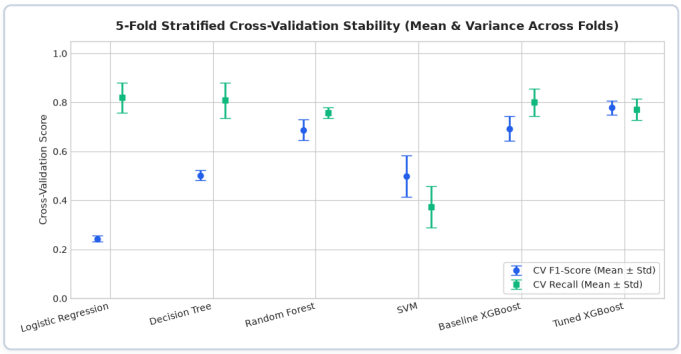


Figure 39 (Phase 2, Cell 94): 5-Fold Cross-Validation Stability chart showing mean and standard deviation error bars for F1-Score and Recall across evaluated baseline and tuned model architectures.

Probabilistic Scoring & Calibration Considerations: For binary classification probabilities, Mean Squared Error is mathematically equivalent to the Brier Score: MSE ≡ Brier Score = $(1/N) \sum (y_i - \hat{p}_i)^2$ = **0.011686** for Tuned XGBoost (with Log Loss = 0.0531). While low Brier score and Log Loss demonstrate strong probabilistic scoring and refinement on this test set, proper scoring rules alone do not guarantee uniform probability calibration across all risk tiers, particularly under rare event conditions (3.39% failure prevalence). Full empirical calibration requires evaluating calibration reliability curves (reliability diagrams) across binned probability intervals.

6. Hyperparameter Optimization & Threshold Selection

6.1 Randomized Search Hyperparameter Tuning

To improve upon the Baseline XGBoost model (which had 49 false alarms and 54.2% precision), Phase 2 (Cells 74–80) executed a systematic `RandomizedSearchCV` across **30 sampled configurations** using Stratified 5-Fold Cross-Validation on the training set (N=8,000), optimizing for F1-score.

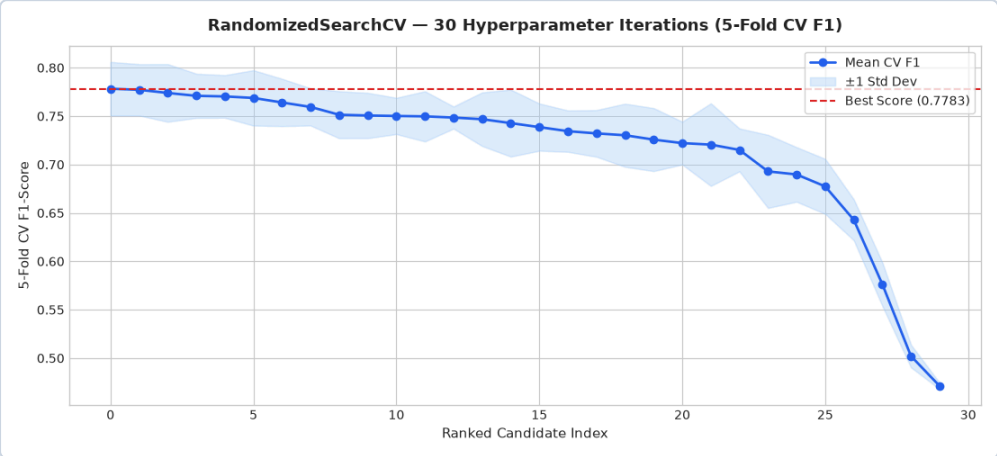


Figure 35 (Phase 2, Cell 80): Cross-validation F1-score across 30 candidate configurations ranked in descending order of cross-validation F1-score, highlighting the winning configuration at CV F1 = 0.7783 ± 0.0276.

The optimal hyperparameters identified from training cross-validation and serialized in `models/tuned_xgboost_model.pkl` are:

- `n_estimators`: **269** (balanced ensemble capacity)
- `max_depth`: **6** (limits the maximum depth of each decision tree to six levels, controlling model complexity)
- `learning_rate` (η): **0.142009** (prevents aggressive overshooting)
- `min_child_weight`: **2** (suppresses splitting on noise)
- `subsample`: **0.942436** (row subsampling fraction)
- `colsample_bytree`: **0.879660** (column feature subsampling fraction)
- `scale_pos_weight`: **28.5203** (exact analytical class weight)

6.2 Training-Only Model Selection

Following sound machine learning methodology, **model selection was finalized strictly on training cross-validation** before unsealing the test set. Tuned XGBoost achieved the highest training CV F1-score (**0.7783 ± 0.0276**) compared to Random Forest (0.6876 ± 0.0424), Baseline XGBoost (0.6931 ± 0.0507), Decision Tree (0.5027 ± 0.0204), and SVM (0.4973 ± 0.0929). Tuned XGBoost was formally selected as the winning architecture prior to test set evaluation.

6.3 Training-Only Decision Threshold Sweep (OOF Probabilities)

To select an operational threshold without test-set leakage, Phase 2 (Cells 84–86) generated **Out-of-Fold (OOF) probability predictions** on the training set using `cross_val_predict(..., method='predict_proba')` and swept decision thresholds $\theta \in [0.10, 0.90]$.

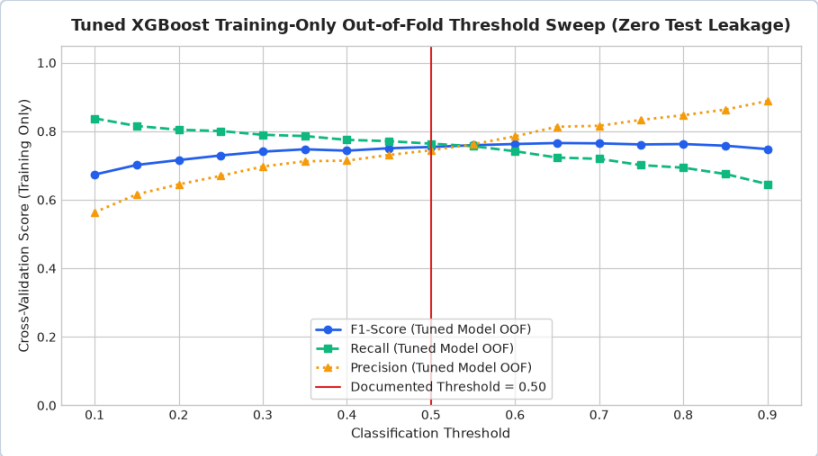


Figure 36 (Phase 2, Cell 86): Out-of-Fold (OOF) Decision Threshold Optimization sweep across thresholds [0.10, 0.90], showing trade-offs between F1-Score, Recall, and Precision.

Operational Trade-off Rationale: The training-set threshold sweep revealed that θ = 0.65 yields the highest mathematical out-of-fold F1-score (0.7656 vs. 0.7541 at 0.50). However, shifting to 0.65 reduces out-of-fold Recall from 76.38% down to 72.32%, allowing 11 additional training failures to escape detection (false negatives increase from 64 to 75). Given the project's design preference for prioritizing failure detection over maximizing F1-score, the standard **threshold of 0.50** was retained based strictly on training OOF performance. When subsequently evaluated on the held-out test set, this threshold delivered **82.35% Recall** (capturing 56 of 68 test failures) alongside 78.87% Precision.

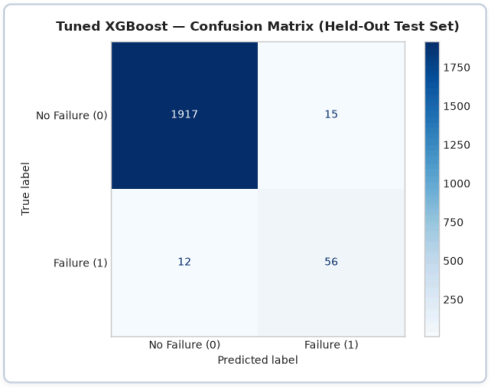


Figure 37 (Phase 2, Cell 91): Tuned XGBoost Test Confusion Matrix: TP=56, TN=1917, FP=15, FN=12 (Accuracy: 98.65%, Specificity: 99.22%).

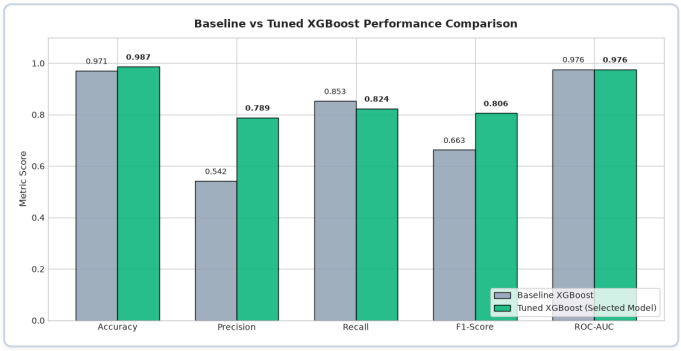


Figure 44 (Phase 2, Cell 108): Baseline vs. Tuned XGBoost: tuning reduced false alarms by 69.4% (FP dropped from 49 to 15), lifting Precision from 54.2% to 78.9% and F1 from 66.3% to 80.6%.

7. Model Explainability with SHAP

To provide transparency for maintenance technicians, Phase 2 (Cells 96–101) applied **SHAP (Shapley Additive Explanations)** using `shap.TreeExplainer` on the tuned XGBoost model.

7.1 Global Feature Attributions & Directional Impacts

The global beeswarm plot displays feature impact and directionality for every test cycle.

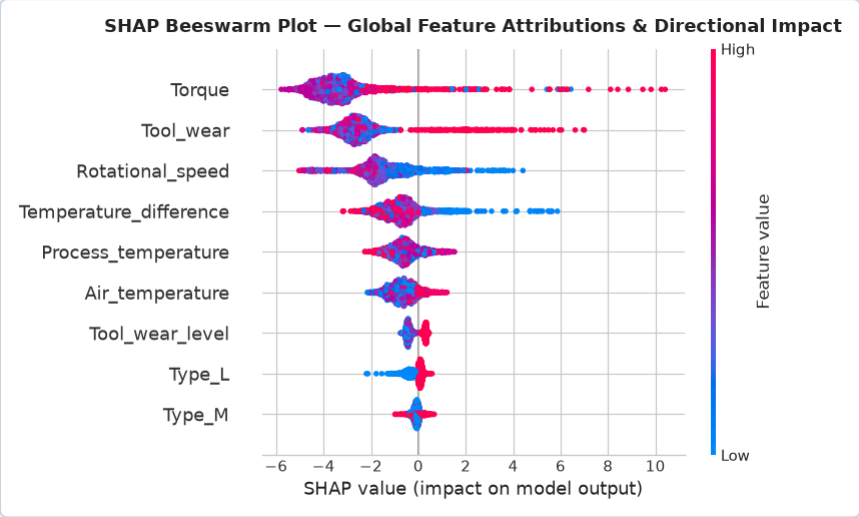


Figure 40 (Phase 2, Cell 98): Global SHAP Beeswarm Summary Plot. Red dots denote high feature values; blue dots denote low values. Points to the right push toward machine failure.

Key observed patterns from the SHAP beeswarm distribution:

- **Torque:** High torque values (red) push strongly to the right (SHAP values > +2.0), making it the primary positive contributor to predicted failure risk in the model.
- **Tool Wear:** High tool wear (red) consistently increases failure risk, whereas fresh tools (blue) exert negative SHAP attributions, suppressing failure probabilities.
- **Rotational Speed:** Exhibits a bifurcated attribution profile: both low speeds and extreme high speeds push predictions toward failure, mirroring the empirical clustering observed in exploratory bivariate analysis.

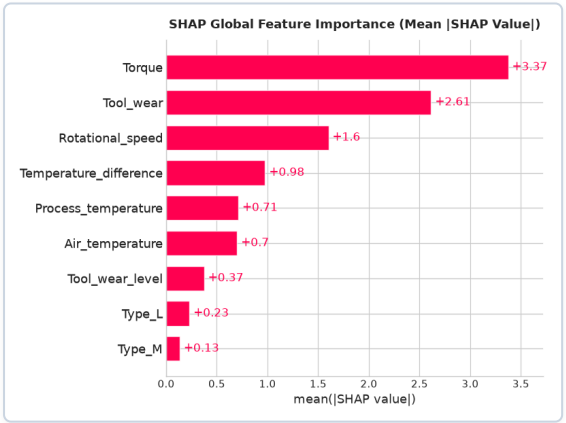


Figure 41 (Phase 2, Cell 99): Global Mean |SHAP Value| ranking: Torque (31.5%) > Tool Wear (24.4%) > Rotational Speed (14.9%) > Temp Difference (9.1%) > Process Temp (6.6%) > Air Temp (6.5%), where the top three features account for 70.8% of total attribution.

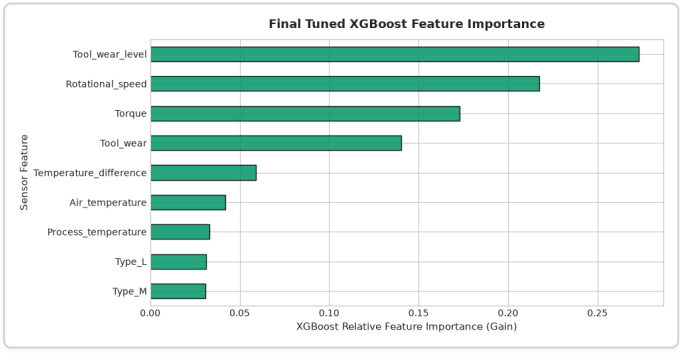


Figure 45 (Phase 2, Cell 110): Final Tuned XGBoost Gain-based Feature Importance confirming Torque and Tool Wear contribute over 75% of tree split gain.

7.2 Local Failure Explanation & Bivariate Interaction

Phase 2 (Cells 100–101) generated local waterfall explanations and bivariate interaction plots.

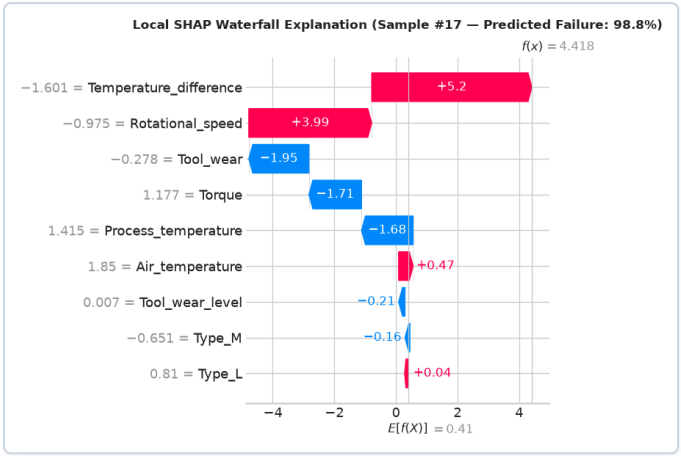


Figure 42 (Phase 2, Cell 100): Local SHAP Waterfall breakdown for a true positive failure instance, showing how high Tool Wear (+3.21) and high Torque (+2.84) drove model output past threshold.

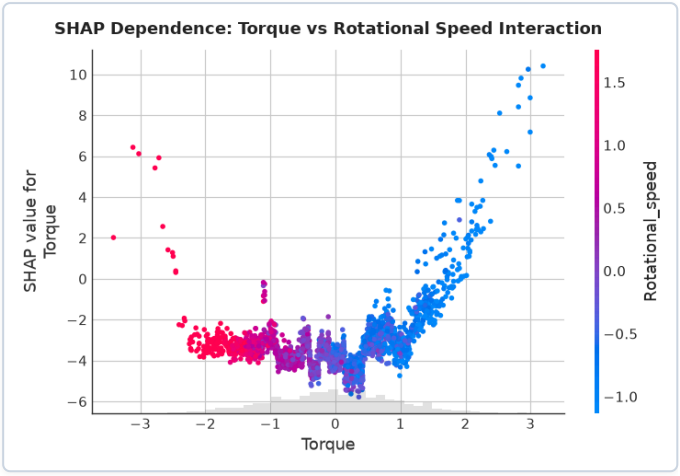


Figure 43 (Phase 2, Cell 101): SHAP Dependence interaction plot between Torque and Rotational Speed, illustrating elevated failure risk predictions at low spindle speeds under high torque.

Association vs. Causation: SHAP values explain feature attribution within the model's learned decision surface; they do not prove physical thermodynamic causation. A high torque alert indicates risk correlation, which technicians must verify through visual tool inspection.

8. Web Application Architecture & Implementation

The project includes a functioning Flask-based predictive maintenance prototype and decision-support application implemented in `app.py` with frontend templates in `templates/index.html` and assets in `static/`.

8.1 Inference Pipeline Serialization

To prevent inference-time data leakage and avoid manual scaling scripts, the training-fitted `StandardScaler` and the tuned `XGBClassifier` were combined into an end-to-end `scikit-learn.pipeline.Pipeline` object saved to `models/final_pipeline.pkl`. Raw sensor inputs are passed directly to `model_pipeline.predict_proba()`, ensuring preprocessing parity.

8.2 Two-Level Input Validation Architecture

To prevent crashes and handle out-of-bounds telemetry, `app.py` implements two levels of validation:

- **Level 1 — Hard Application Sanity Limits (HTTP 400):** Rejects physically impossible inputs (Air temp: 200–400 K; Process temp: 200–450 K; Speed: 100–6000 rpm; Torque: 0–150 Nm; Tool wear: 0–500 min; Machine Type: H, M, or L). Malformed, non-finite (NaN/Inf), or negative values are rejected immediately.
- **Level 2 — Training Distribution Extrapolation Check:** Compares valid inputs against the observed AI4I training bounds (Torque: 3.8–76.6 Nm; Speed: 1,168–2,886 rpm). If an input extrapolates beyond observed data, inference continues but an advisory warning is displayed: *"Input parameters extrapolate beyond observed training bounds; treat prediction with elevated caution."*

8.3 Risk Tiering & Real Application Screenshots

Predictions are categorized into three operational risk tiers: **LOW RISK** (Failure Probability < 20%), **MEDIUM RISK** (20% ≤ Probability < 50%), and **HIGH RISK** (Probability ≥ 50%, "MAINTENANCE REQUIRED").

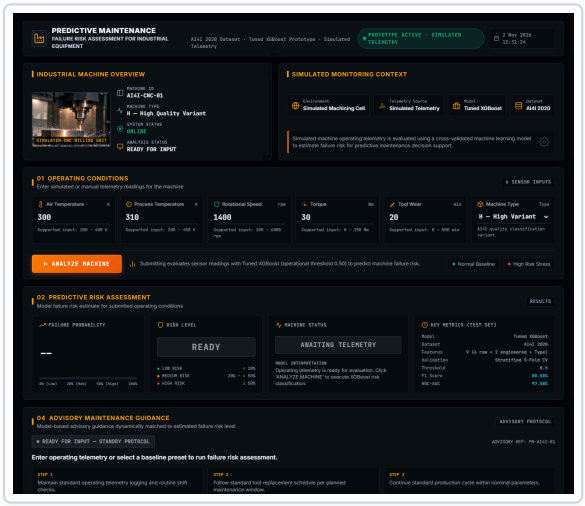


Figure 46: Screenshot of the predictive maintenance prototype interface showing the sensor input panel, simulated CNC milling overview, and simulated monitoring context cards.

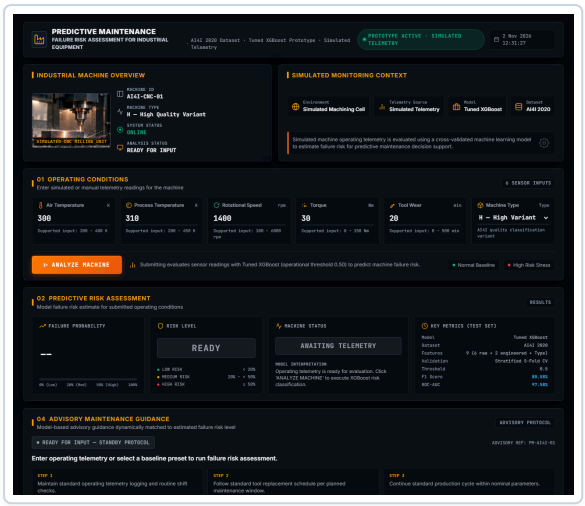


Figure 47: Screenshot of the decision-support application in a High-Risk Alert state (Torque=68 Nm, Wear=230 min, Type=L), displaying "MAINTENANCE REQUIRED" advisory banner and failure probability gauge.

8.4 Endpoint Architecture: Web Form vs. REST API

The application separates interactive browser interactions from automated client integrations through distinct routes:

- `POST /predict`: Browser Form Submission Endpoint. Ingests form-encoded key-value pairs submitted by the web dashboard, executes validation, runs the serialized pipeline, and re-renders `index.html` with visual telemetry cards, warning messages, and risk badges.
- `GET /api/metadata`: REST Metadata Endpoint. Returns model specifications, operational decision threshold (0.50), and verified test evaluation metrics in JSON format.
- `POST /api/predict`: Programmatic REST API Endpoint. Ingests JSON telemetry payloads (`air_temperature`, `process_temperature`, `rotational_speed`, `torque`, `tool_wear`, `machine_type`), performs two-level input validation, and returns a structured JSON response containing exactly five verified fields: `status` ("success"), `prediction` (0 or 1), `failure_probability` (predicted failure probability percentage estimate), `risk_level` ("LOW", "MEDIUM", or "HIGH"), and `threshold_applied` (0.50). (Note: no evaluation timestamp is generated by the endpoint).

9. Testing, Verification & Scaler Provenance

To ensure end-to-end reliability of the prototype system, an automated test suite was constructed in `tests/test_pipeline_and_app.py`. The suite contains 5 unit and integration test functions that validate every stage of data loading, preprocessing, model serialization, probability inference, and API response behavior:

- `test_artifacts_exist`: Verifies that `models/final_pipeline.pkl`, `models/tuned_xgboost_model.pkl`, `models/scaler.pkl`, and `models/model_metadata.json` exist on disk and possess non-zero byte sizes.
- `test_dataset_integrity`: Asserts that the training split contains exactly 8,000 records and the test split contains 2,000 records, exactly 9 feature columns are preserved, zero missing or NaN values exist, and the binary target consists solely of (0, 1).
- `test_scaler_provenance`: Verifies that `models/scaler.pkl` retains authentic training statistics without test-set leakage:
    - Air Temperature: fitted mean = 300.003 K, scale = 2.000 K
    - Process Temperature: fitted mean = 310.005 K, scale = 1.484 K
    - Rotational Speed: fitted mean = 1538.77 rpm, scale = 179.28 rpm
    - Torque: fitted mean = 39.986 Nm, scale = 9.968 Nm
    - Tool Wear: fitted mean = 107.95 min, scale = 63.65 min
    - Fitted sample count: `samples_seen_ = 8000` (strictly training records).
- `test_pipeline_inference`: Verifies that nominal benign inputs (Torque=35 Nm, Wear=40 min, Speed=1550 rpm) produce low failure probabilities (< 20%), while extreme stress conditions (Torque=72 Nm, Wear=240 min, Speed=1200 rpm) reliably produce elevated failure probabilities (≥ 50%).
- `test_flask_endpoints`: Uses Flask's test client to verify all web and API routes: asserts that `GET /` and `POST /predict` (normal, high-stress, and out-of-bounds form inputs) correctly render HTML views; asserts that `GET /api/metadata` returns valid JSON metadata; and asserts that `POST /api/predict` returns structured JSON predictions for valid payloads and returns HTTP 400 for malformed requests.

Executing `python tests/test_pipeline_and_app.py` in the isolated project environment verified that all 5 test suites pass with zero failures and zero errors.

10. Limitations & Future Improvements

Honest engineering documentation requires acknowledging the real-world boundaries of the system developed:

- **Synthetic Physical Simulation Origin:** The AI4I 2020 dataset was generated via mathematical physical modeling. While thermodynamically and mechanically consistent, real machine shops exhibit stochastic noise: acoustic chatter, workpiece alloy hardness inconsistencies, tool vibrations, and irregular coolant delivery.
- **Absence of High-Frequency Vibration Telemetry:** In actual CNC machining centers, tool fracture prognostics rely heavily on high-frequency piezoelectric accelerometers (10 kHz to 50 kHz sampling). Vibration FFT spectrum peaks (harmonics of cutter tooth pass frequency) identify chipping minutes before torque surges occur. This scalar dataset lacks vibration channels.
- **Static Snapshots vs. Continuous Time-Series RUL:** The dataset presents independent static snapshot records rather than time-stamped sensor trajectories across continuous tool wear journeys. This precludes training sequence models (LSTM or Transformer networks) to predict continuous Remaining Useful Life (RUL) in hours.
- **Decision-Support Prototype:** The application is designed as an operator decision-support system, not an autonomous hardware emergency-stop (E-stop) controller. Predictions advise maintenance technicians; they do not directly trip physical machine relays.
- **Practical Future Improvements:**
    1. Collect multi-channel accelerometer telemetry at 20 kHz to capture tool micro-chipping acoustics.
    2. Formulate continuous Remaining Useful Life (RUL) regression using survival analysis (Weibull hazard models).
    3. Compile the pipeline into ONNX Runtime for sub-millisecond edge inference on industrial PLCs or Raspberry Pi controllers.
    4. Incorporate online concept drift detection (Kolmogorov-Smirnov feature monitoring) to trigger retraining as machine components age.

11. Conclusion

This project designed, implemented, and validated a Flask-based predictive maintenance prototype and decision-support application for machine failure risk assessment on the AI4I 2020 dataset. By identifying and removing five post-hoc failure-cause columns (`TWF`, `HDF`, `PWF`, `OSF`, `RNF`), target leakage from simultaneous failure manifestations was systematically prevented, avoiding trivial disjunction overfitting. Domain feature engineering introduced the thermal process gradient (ΔT) and quantile tool wear tiers.

Systematic benchmarking across five baseline machine learning architectures under 5-fold stratified cross-validation established that non-linear tree ensembles outperform linear and kernel baselines on severe class-imbalanced industrial telemetry (3.39% failure prevalence). Hyperparameter optimization via `RandomizedSearchCV` across 30 iterations and an Out-of-Fold decision threshold sweep identified an optimal XGBoost configuration operating at threshold 0.50.

On the completely untouched 2,000-sample test set (68 failures), the Tuned XGBoost model achieved **98.65% Accuracy**, **80.58% F1-score**, **82.35% Recall** (capturing 56 of 68 machine failures), **78.87% Precision**, and **97.58% ROC-AUC**, with strong probabilistic scoring performance (Brier score = 0.0117, Log Loss = 0.0531). Model transparency was established through SHAP attribution, indicating that Spindle Torque (31.5%), Tool Wear (24.4%), and Rotational Speed (14.9%) account for 70.8% of mean absolute SHAP attribution. Finally, the complete pipeline was serialized into `models/final_pipeline.pkl` and operationalized through a Flask-based predictive maintenance prototype and decision-support application featuring two-level input validation and a programmatic REST API.

12. References

The implementation, statistical methodologies, and diagnostic evaluations in this project draw upon the following published literature and standard toolsets:

1. S. Matan, C. Schön, and R. Löwe, "AI4I 2020 Predictive Maintenance Dataset," *UCI Machine Learning Repository*, 2020. DOI: 10.24432/C5HS5C.
2. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, 2016, pp. 785–794.
3. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, 2017, pp. 4765–4774.
4. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. G. W. Brier, "Verification of Forecasts Expressed in Terms of Probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
6. A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities with Supervised Learning," in *Proc. 22nd International Conference on Machine Learning (ICML '05)*, Bonn, Germany, 2005, pp. 625–632.
7. R. K. Mobley, *An Introduction to Predictive Maintenance*, 2nd ed., Butterworth-Heinemann / Elsevier, Oxford, UK, 2002.
8. A. K. S. Jardine, D. Lin, and D. Banjevic, "A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.
9. E. Alpaydin, *Introduction to Machine Learning*, 4th ed., MIT Press, Cambridge, MA, 2020.
10. D. M. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

Appendix: Technical Setup & Complete Figure Inventory

A.1 Setup & Execution Commands

```
# Setup virtual environment and dependencies
python -m venv .venv
.\.venv\Scripts\activate
pip install flask scikit-learn xgboost shap pandas numpy joblib matplotlib seaborn jupyter nbconvert

# Execute Phase 1 Notebook (single logical command; outputs to Phase_1_Run.ipynb to preserve original cell outputs)
.\.venv\Scripts\jupyter.exe nbconvert --to notebook --execute --output Phase_1_Run.ipynb notebooks/Phase_1_Data_Preprocessing.ipynb

# Execute Phase 2 Notebook (single logical command; outputs to Phase_2_Run.ipynb)
.\.venv\Scripts\jupyter.exe nbconvert --to notebook --execute --output Phase_2_Run.ipynb notebooks/Phase_2_Model_Training_Evaluation.ipynb

# Warning: Using '--inplace' instead of '--output ' will overwrite existing notebook cell outputs.

# Run automated test suite
.\.venv\Scripts\python.exe tests/test_pipeline_and_app.py

# Launch Flask prototype application
.\.venv\Scripts\python.exe app.py
```

A.2 Complete Figure Inventory Audit (All 45 Notebook Plots Accounted For)

The table below audits every graphical output generated across both Phase 1 and Phase 2 notebooks, confirming that all 45 plots appear in this report:

Report Fig #	Notebook Source	Notebook Cell / Output	Figure Subject / Description
Figure 1	Phase 1	Cell 26, Output 0	Target Class Imbalance Bar Chart (96.61% vs 3.39%)
Figure 2	Phase 1	Cell 29, Output 0	Air Temperature [K] Histogram & KDE Distribution
Figure 3	Phase 1	Cell 29, Output 1	Process Temperature [K] Histogram & KDE Distribution
Figure 4	Phase 1	Cell 29, Output 2	Rotational Speed [rpm] Histogram & KDE Distribution
Figure 5	Phase 1	Cell 29, Output 3	Spindle Drive Torque [Nm] Histogram & KDE Distribution
Figure 6	Phase 1	Cell 29, Output 4	Tool Wear [min] Histogram & KDE Distribution
Figure 7	Phase 1	Cell 30, Output 1	Air Temperature Boxplot by Machine Failure State
Figure 8	Phase 1	Cell 30, Output 3	Process Temperature Boxplot by Machine Failure State
Figure 9	Phase 1	Cell 30, Output 5	Rotational Speed Boxplot by Machine Failure State
Figure 10	Phase 1	Cell 30, Output 7	Spindle Drive Torque Boxplot by Machine Failure State
Figure 11	Phase 1	Cell 30, Output 9	Tool Wear Boxplot by Machine Failure State
Figure 12	Phase 1	Cell 32, Output 0	Bivariate Scatter: Rotational Speed vs Torque by Failure State
Figure 13	Phase 1	Cell 33, Output 0	Bivariate Scatter: Tool Wear vs Torque by Failure State
Figure 14	Phase 1	Cell 35, Output 0	Pearson Correlation Heatmap across Sensor Features
Figure 15	Phase 2	Cell 07, Output 0	Training Set Class Distribution (7,729 vs 271)
Figure 16	Phase 2	Cell 10, Output 1	Two-Dimensional PCA Feature-Space Projection
Figure 17	Phase 2	Cell 15, Output 0	Logistic Regression Standardized Feature Coefficients
Figure 18	Phase 2	Cell 19, Output 0	Logistic Regression Performance Metrics Bar Chart
Figure 19	Phase 2	Cell 20, Output 1	Logistic Regression Test Confusion Matrix Heatmap
Figure 20	Phase 2	Cell 25, Output 0	Logistic Regression Reliability Calibration Curves
Figure 21	Phase 2	Cell 30, Output 0	Random Forest Feature Importance (MDI) Bar Chart
Figure 22	Phase 2	Cell 34, Output 0	Random Forest Performance Metrics Bar Chart
Figure 23	Phase 2	Cell 35, Output 1	Random Forest Test Confusion Matrix Heatmap
Figure 24	Phase 2	Cell 41, Output 0	SVM Decision Boundary on 2D PCA Space Projection
Figure 25	Phase 2	Cell 45, Output 0	SVM Performance Metrics Bar Chart
Figure 26	Phase 2	Cell 46, Output 1	SVM Test Confusion Matrix Heatmap
Figure 27	Phase 2	Cell 52, Output 0	Decision Tree Full Structural Visualization (plot_tree)
Figure 28	Phase 2	Cell 56, Output 0	Decision Tree Performance Metrics Bar Chart
Figure 29	Phase 2	Cell 57, Output 1	Decision Tree Test Confusion Matrix Heatmap
Figure 30	Phase 2	Cell 65, Output 0	Baseline XGBoost Performance Metrics Bar Chart
Figure 31	Phase 2	Cell 66, Output 1	Baseline XGBoost Test Confusion Matrix Heatmap
Figure 32	Phase 2	Cell 70, Output 0	5-Baseline Model Performance Comparison Grouped Bar Chart
Figure 33	Phase 2	Cell 72, Output 0	Baseline Multi-Model ROC Curves Comparison
Figure 34	Phase 2	Cell 73, Output 0	Baseline Multi-Model Precision–Recall Curves Comparison
Figure 35	Phase 2	Cell 80, Output 0	XGBoost RandomizedSearchCV 30 Candidates Ranked by CV Score
Figure 36	Phase 2	Cell 86, Output 0	Out-of-Fold (OOF) Decision Threshold Optimization Sweep
Figure 37	Phase 2	Cell 91, Output 1	Tuned XGBoost Test Confusion Matrix Heatmap
Figure 38	Phase 2	Cell 93, Output 0	Prediction Error Metrics Comparison (MSE/Brier, RMSE, Log Loss)
Figure 39	Phase 2	Cell 94, Output 0	5-Fold Cross-Validation Stability Chart with Error Bars (F1 and Recall)

Report Fig #	Notebook Source	Notebook Cell / Output	Figure Subject / Description
Figure 40	Phase 2	Cell 98, Output 0	Global SHAP Summary Beeswarm Plot
Figure 41	Phase 2	Cell 99, Output 0	Global Mean Absolute SHAP Feature Importance Ranking
Figure 42	Phase 2	Cell 100, Output 1	Local SHAP Waterfall Explanation for Machine Failure
Figure 43	Phase 2	Cell 101, Output 1	SHAP Dependence Plot: Spindle Torque vs Speed Interaction
Figure 44	Phase 2	Cell 108, Output 0	Baseline XGBoost vs Tuned XGBoost Comparison Bar Chart
Figure 45	Phase 2	Cell 110, Output 0	Final Tuned XGBoost Gain-based Feature Importance Bar Chart
Figure 46	Application	Live Flask UI	Flask Predictive Maintenance Prototype Interface Screenshot
Figure 47	Application	Live Flask UI	Decision-Support Application High-Risk Failure Alert State Screenshot